Understanding Linux File Access Control

Interpreting File and Directory Permissions

Overview

Linux uses a permission model that controls how users and groups interact with files and directories. By understanding these permissions, you can determine who can read, modify, or execute a file, and who can manage the contents of a directory.

How Linux File Permissions Work

Each file and directory has:

- **One owner (a user)** – usually the creator
- **One owning group** – usually the user's primary group
- **Three permission sets:**
    - **User** (owner)
    - **Group** (owning group)
    - **Other** (everyone else on the system)

Permissions are applied in this order of priority:

1. **User permissions**
2. **Group permissions**
3. **Other permissions**

The most specific match wins.
So if a user owns the file, only the **user** permissions matter – even if they are also part of the file's group.

Collaboration Example

Imagine two users, *alice* and *bob*:

- alice belongs to **alice** and **web** groups
- bob belongs to **bob**, **wheel**, and **web** groups

For shared work, files should belong to the **web** group, and group permissions should allow both users to read/write those files.

Permission Types

Linux assigns three basic permission types:

Permission	Symbol	What it means for files	What it means for directories
Read	r	View the file's contents	List names of files inside the directory
Write	w	Modify or overwrite file contents	Create/delete/rename files inside the directory
Execute	x	Run the file as a program/script	Enter the directory (`cd`), access files inside if you know the filename

Important Notes

- A directory that is “read-only” to users usually still needs **r + x** to allow listing its contents.
- **Read only without execute:** user can list filenames but cannot access their metadata or open them.
- **Execute only without read:** user cannot list filenames, but can access a file **if they already know its name**.
- Deleting a file depends **on the directory's write permission**, not the file’s permission.
- The **sticky bit** restricts deletion inside shared directories (explained later).

Comparison to Windows NTFS

- Linux permissions apply only to the exact file/directory; they do **not cascade** into subdirectories.
- “Read” in Linux ≈ “List folder contents” in Windows.
- “Write” in Linux ≈ ability to modify and delete, similar to “Modify” in Windows.
- With sticky bit + write permission, Linux behaves closer to Windows “Write only owner can delete.”
- The **root** user can bypass normal file permissions, but **SELinux** rules can still restrict root (covered in RH134).

Viewing Permissions and Ownership

Use **ls -l** to view detailed file permissions:

```
ls -l test
```

Example output:

```
-rw-rw-r--. 1 student student 0 Mar  8 17:36 test
```

To view details of a directory itself (not its contents):

```
ls -ld /home
```

Example output:

```
drwxr-xr-x. 5 root root 4096 Feb 31 22:00 /home
```

Understanding the Output

- **First character**: file type
  - `-` regular file
  - `d` directory
  - `l` symbolic link
  - `c` character device
  - `b` block device
  - `p` named pipe
  - `s` socket
- **Next nine characters**: permissions split into three groups
  - user (owner)
  - group
  - others

Example: `rw-rw-r--`

- `rw-` → owner can read and write
- `rw-` → group can read and write
- `r--` → others can only read

After the permissions and link count:

- First name = file owner
- Second name = owning group
-

Permission Behavior Examples

Assume four users with these group memberships:

User	Groups
operator1	operator1, consultant1
database1	database1, consultant1
database2	database2, operator2
contractor1	contractor1, operator2

Inside directory `dir`, the files have these permissions:

```
drwxrwxr-x. 2 database1 consultant1  4096 Mar  4 10:23 .
drwxr-xr-x. 10 root      root        4096 Mar  1 17:34 ..
-rw-rw-r--. 1 operator1  operator1    1024 Mar  4 11:02 app1.log
-rw-r--rw-. 1 operator1  consultant1  3144 Mar  4 11:02 app2.log
-rw-rw-r--. 1 database1  consultant1 10234 Mar  4 10:14 db1.conf
-rw-r-----. 1 database1  consultant1  2048 Mar  4 10:18 db2.conf
```

Why certain users can or can't do things

Observation	Why it happens
operator1 can edit `db1.conf`	operator1 belongs to *consultant1*, and that group has r+w.
database1 can edit `db2.conf`	database1 owns the file → owner r+w applies.
operator1 can read but not edit `db2.conf`	consultant1 group only has read permission.

database2 and contractor1 cannot read `db2.conf`	“Other” permissions lack read/write.
Only operator1 can modify `app1.log`	Only operator1 is in the operator1 group, which has write permission.
database2 can edit `app2.log`	“Other” permissions allow write access.
database1 can read but not write `app2.log`	database1 is in consultant1, and group permissions override “other.” Group has only read.
database1 can delete `app1.log` **and** `app2.log`	database1 has write access to the directory `dir`, allowing file deletion regardless of file permissions.

References

- **man ls**
- **info coreutils** → Section 13: Changing File Attributes

Manage File System Permissions from the Command Line

Adjusting File Ownership and Access Rights

Goal

Learn how to modify permissions and ownership of files and directories using command-line utilities.

Changing Permissions on Files and Directories

The primary tool for adjusting permissions in Linux is **chmod**.

- Its name comes from **“change mode,”** where *mode* refers to the permission settings on a file or directory.
- The command accepts a **permission specification** followed by one or more files or directories.
- Permissions can be set using **symbolic notation** (letters) or **octal notation** (numbers).

1. Modifying Permissions Using Symbolic Notation

The symbolic format uses three components:

```
chmod <who><operation><which> file|directory
```

Who (user class)

Indicates whose permissions you’re modifying. If omitted, the default is **a** (all).

Symbol	Applies to	Meaning
u	user	The file owner
g	group	Members of the file’s group
o	other	Everyone else
a	all	u + g + o

Operation (what to do)

Symbol	Meaning	Description
+	add	Adds the listed permission(s)
-	remove	Removes the listed permission(s)
=	set exactly	Replaces the entire permission set for that class

Which (permission type)

Symbol	Permission	Meaning
r	read	View file contents / list directory entries
w	write	Modify file or directory contents
x	execute	Run a file / enter a directory
X	conditional execute	Apply execute only to directories or to files that already have at least one execute bit

How symbolic permissions behave

- You can change only specific bits without rewriting everything.
- `+` adds, `-` removes, and `=` overwrites.
- `X` is useful for recursively fixing directory execute bits without accidentally making regular files executable.

Symbolic Examples

Remove read/write for group and other:
```
chmod go-rw document.pdf
```

Give everyone execute permission on a script:
```
chmod a+x myscript.sh
```

Recursively give the group full access to a directory:
```
chmod -R g+rwx /home/user/myfolder
```

Recursively add read/write for group, and only add execute where appropriate:
```
chmod -R g+rwX demodir
```

2. Modifying Permissions Using Octal Notation

The octal method assigns each permission a numeric value:

Permission	Value
read (r)	4
write (w)	2
execute (x)	1

Add these numbers together to get the permission digit.

Structure:

```
chmod ### file|directory
```

Each digit represents:

1. user
2. group
3. other

Example interpretations:

- **644** → user: rw-, group: r--, other: r--
- **750** → user: rwx, group: r-x, other: ---

Octal Examples

Set `rw-r--r--`:

```
chmod 644 sample.txt
```

Set `rwxr-x---`:

```
chmod 750 sampledir
```

Experienced administrators often prefer octal notation because it's exact and efficient, especially for scripting.

Changing File and Directory Ownership

By default:

- A file belongs to the **user** who created it.
- The file's **group** is usually the creator's primary group.

To give other users or groups access, ownership may need to be adjusted.

Ownership Permissions

- **Only root** can change file ownership from one user to another.
- The **file owner** or **root** can change the group, but a regular user can only assign a group if they belong to it.

Using `chown` (change owner)

Change the user owner:

```
chown student app.conf
```

Change owner recursively:

```
chown -R student Pictures
```

Change only the group:

```
chown :admins Pictures
```

Change both user and group:

```
chown visitor:guests Pictures
```

Using `chgrp` (change group)

An alternative to `chown :group` for modifying group ownership:

```
chgrp admins Pictures
```

Important Note on Syntax

Some systems allow:
`chown owner.group file`

This **should be avoided** because a period (.) is a valid character in usernames, causing ambiguity.
Always use a colon (`owner:group`) to avoid misinterpretation.

References

- `ls(1)` man page
- `chmod(1)` man page
- `chown(1)` and `chgrp(1)` man pages

Manage Default Permissions and Special File Access Controls

Understanding umask, setuid, setgid, and the Sticky Bit

Objectives

- Control the default permissions applied to newly created files and directories
- Explain how special permission bits modify normal access behavior
- Use special permissions to enforce group ownership inheritance within shared directories

Special Permission Bits

In addition to the regular **user**, **group**, and **other** permissions, Linux provides a fourth category known as **special permissions**. These bits add behavior that cannot be achieved through normal read/write/execute permissions alone.

The three special permission types are:

Special Permission	Symbol	Effect on Files	Effect on Directories
setuid	u+s	File runs with the **owner's UID**	No effect
setgid	g+s	File runs with the **group's GID**	Newly created files inherit the directory's group
sticky bit	o+t	No effect	Only file owner (or root) can delete files

How Special Permissions Work

1. setuid (u+s)

When set on an executable, the program runs using the **file owner's privileges**, not the privileges of the person running it.
Example: the `passwd` command

```
ls -l /usr/bin/passwd
-rwsr-xr-x. 1 root root 35504 Jul 16 2010 /usr/bin/passwd
```

- The **s** in place of the owner's **x** indicates setuid is active.
- If execute is missing, it shows as **S**, meaning setuid is set but the execute bit isn't.

2. setgid (g+s)

On directories

Files created inside inherit the **directory's group**, not the creator's primary group.
Example:

```
ls -ld /run/log/journal
drwxr-sr-x. 3 root systemd-journal 60 May 18 09:15 /run/log/journal
```

This is commonly used in collaborative group directories.

On executables

The program runs with the group owner's permissions.

Example:

```
ls -l /usr/bin/locate
-rwx--s--x. 1 root slocate 47128 Aug 12 17:17 /usr/bin/locate
```

The **s** appears where group execute normally appears.

3. Sticky Bit (o+t)

Used only on directories.
A directory with the sticky bit allows:

- anyone with write access to **create** files

- but only the **file owner** (or root) can delete files

Example:
```
ls -ld /tmp
drwxrwxrwt. 39 root root 4096 Feb 8 20:52 /tmp
```

The trailing **t** indicates the sticky bit. If execute for others is missing, it appears as **T**.

Setting Special Permissions

Symbolic Method

- **setuid:** `u+s`
- **setgid:** `g+s`
- **sticky bit:** `o+t`

Octal Method

Special bits use a **fourth leading digit** in the permission value:

- setuid → **4***
- setgid → **2***
- sticky → **1***

Examples

Add setgid using symbolic notation:
```
chmod g+s example
```

Remove setuid:
```
chmod u-s example
```

Set setgid plus `rwx` for user and group, no access for others:
```
chmod 2770 example
```

Remove setgid using numeric syntax:
```
chmod 0770 example
```

Default Permissions and umask

When a new file or directory is created, its initial permissions depend on:

1. **The type of object** (file or directory)
2. **The current umask value**

Default base permissions before umask is applied:

- **Regular files:** `0666` (rw-rw-rw-) → execute bits intentionally disabled
- **Directories:** `0777` (rwxrwxrwx)

The system then applies the **umask**, which *subtracts* permissions.

Understanding umask

A umask is an octal mask where:

- any bit set to `1` removes that permission
- bits set to `0` leave the permission intact

Example:

```
umask 0002
```

- removes write for **other**
- leaves user and group permissions unchanged

Show current umask:

```
umask
```

Change it:

```
umask 077
```

User-level overrides can be placed in:

- `~/.bashrc`
- `~/.bash_profile`

System defaults are in:

- `/etc/login.defs`
- `/etc/bashrc`

Important RHEL 9 Behavior

- RHEL 9.0 and 9.1:
  - Login shells default to `0022`
  - Non-login shells may use `0002` for users with UID ≥ 200 and matching username/group
- Future versions will standardize on **0022** for all shell types
- Change tracking: https://bugzilla.redhat.com/show_bug.cgi?id=2062601

How umask Affects File Creation

Assume:
umask = 0022

File creation

Base permissions = 0666
 Subtract umask:
0666
-0022

0644 → rw-r--r--

Example:
touch default.txt
ls -l default.txt
-rw-r--r--. 1 user user 0 May 9 01:54 default.txt

Directory creation

Base permissions = 0777
0777
-0022

0755 → rwxr-xr-x

Example:
mkdir default
ls -ld default
drwxr-xr-x. 2 user user 0 May 9 01:54 default

More umask Examples

umask 0

No restrictions:
Files → 666
 Directories → 777
umask 0
touch zero.txt   → rw-rw-rw-
mkdir zero   → rwxrwxrwx

umask 007

Blocks all permissions for “other”:
umask 007
touch seven.txt → rw-rw----
mkdir seven   → rwxrwx---

umask 027

User gets full access; group gets restricted access; others get none.

```
umask 027
touch two-seven.txt  → rw-r-----
mkdir two-seven      → rwxr-x---
```

Changing System-Wide Default umask

In RHEL 9, `/etc/login.defs` controls default umask for login shells. To override defaults for interactive non-login shells, create:

`/etc/profile.d/local-umask.sh`

Example:

```
# Overrides default umask configuration
if [ $UID -gt 199 ] && [ "$(id -gn)" = "$(id -un)" ]; then
    umask 007
else
    umask 022
fi
```

To force **umask 022** for all users:

```
umask 022
```

Remember: the umask stays active for the current shell session until logout or until manually changed.

References

- bash(1)
- ls(1)
- chmod(1)
- umask(1)

Manage SELinux Security

Goal

Strengthen and control server security by using SELinux.

Objectives

By the end, you should be able to:

- Describe how SELinux protects system resources, switch the current SELinux mode, and configure the system's default SELinux mode.
- Manage SELinux file and directory contexts using `semanage fcontext` and apply those contexts with `restorecon`.
- Enable and disable SELinux policy features using Booleans:
  - Toggle Booleans with `setsebool`
  - Manage persistent Boolean values with `semanage boolean -l`
  - Use man pages ending in `_selinux` to understand what each Boolean controls
- Use SELinux analysis tools such as `sealert` to review and troubleshoot SELinux-related issues.

Main Topics

- Change the SELinux Enforcement Mode (with guided practice)
- Control SELinux File Contexts (with guided practice)
- Adjust SELinux Policy with Booleans (with guided practice)
- Analyze and Fix SELinux Issues (with guided practice)

Change the SELinux Enforcement Mode

Objective

Explain how SELinux protects resources, adjust the current SELinux mode, and configure the default SELinux mode on a system.

SELinux Architecture - High-Level View

Security-Enhanced Linux (SELinux) adds a strong, policy-driven security layer on top of traditional Linux permissions. It allows very fine-grained control over access to files, network ports, and other system resources.

- Regular file permissions (user/group/other) control **who** can read, write, or execute.
- They do **not** control **how** a file is used once access is granted.

Example:
 If a user has write access to a structured data file, any program that user runs might modify the file, even if that file is supposed to be edited only by a specific trusted application. Traditional permissions cannot prevent misuse like that.

SELinux solves this with **policies**:

- Each application or service gets a **targeted policy** describing exactly what it is allowed to do.
- Policies define labels (contexts) for binaries, configuration files, data files, and ports.
- These labels and rules control which process can access which resources and in what way.

SELinux Usage Model

SELinux enforces **allow rules** between subjects (processes) and objects (files, dirs, ports, etc.):

- If a specific access is **not allowed** in the policy, it is **blocked**.
- Even poorly designed or vulnerable applications are constrained by SELinux rules.

Applications with a policy run inside a **confined domain**. Processes without a policy usually run in an **unconfined** domain and are not protected by SELinux constraints.

SELinux Modes

SELinux has three operational modes:

1. **Enforcing**
   - Policies are loaded and enforced.
   - Access that violates policy is denied and logged.
   - This is the default mode on Red Hat Enterprise Linux.
1. **Permissive**
   - Policies are loaded and checked.
   - Violations are **not blocked**, but are logged as warnings.
   - Useful for testing and tuning policy without breaking services.
1. **Disabled**
   - SELinux is completely off.
   - No enforcement and no SELinux auditing.
   - Strongly discouraged in production.

Important (RHEL 9 behavior)

Starting with **RHEL 9**:

- SELinux can be fully disabled **only** with the kernel parameter `selinux=0` at boot.
- Setting `SELINUX=disabled` in `/etc/selinux/config` **no longer** fully disables SELinux.

If you set `SELINUX=disabled` in the config file on RHEL 9:

- SELinux starts enforcing **without any policy loaded**.
- With no policy rules, everything is denied by default.
- This is intentional, to prevent attackers from turning SELinux off in a meaningful way.

DAC vs MAC - Basic Concepts

Traditional Linux permissions are **Discretionary Access Control (DAC)**:

- Admins and users can change permissions on their own files at will.
- Access is controlled mainly by user IDs, group IDs, and `rwx` bits.

SELinux adds **Mandatory Access Control (MAC)**:

- Policies define what is allowed at a system level.
- These rules apply to all users and processes, regardless of file owner decisions.
- DAC must allow access **and** SELinux must allow access; otherwise, access is denied.

Example scenario:
 A web server process (running as `apache` or `httpd`) gets compromised by a remote attacker.

- DAC might allow that process to access `/var/www/html`, `/tmp`, `/var/tmp`, etc.
- SELinux policy can strictly limit the web server process to specific labeled content and ports, blocking access to unrelated areas even if file permissions would allow it.

SELinux Contexts and Types

SELinux assigns every significant object (file, process, directory, port) a **context**, which includes fields such as:

- SELinux user
- Role
- Type
- Security level

The **type** field is most important in targeted policy and typically ends in `_t`.

Examples:

- Web server process: `httpd_t`
- Web content: `httpd_sys_content_t`
- Temporary files: `tmp_t`
- HTTP ports: `http_port_t`
- MariaDB process: `mysqld_t`
- MariaDB database files: `mysqld_db_t`

Example of type-based rules

- The Apache process labeled `httpd_t` is allowed to read content labeled `httpd_sys_content_t` (e.g., `/var/www/html`).
- The Apache domain has **no allow rule** for files labeled `tmp_t`, so `/tmp` and `/var/tmp` are off-limits at the SELinux level.
- A MariaDB process labeled `mysqld_t` can access `mysqld_db_t`-labeled files but **not** `httpd_sys_content_t` files, by default.

Viewing SELinux Contexts

Most standard commands support a `-Z` option to show SELinux labels:

- `ps -Z`
- `ls -Z`
- `cp -Z`
- `mkdir -Z`

Examples:

```
# Show all processes with their labels
ps axZ

# Start Apache and inspect its SELinux domain
systemctl start httpd
ps -ZC httpd
```

Sample output fragment:

```
system_u:system_r:httpd_t:s0  1550 ?  00:00:00 httpd
system_u:system_r:httpd_t:s0  1551 ?  00:00:00 httpd
...
```

Check web content labels:

```
ls -Z /var/www
system_u:object_r:httpd_sys_script_exec_t:s0 cgi-bin
system_u:object_r:httpd_sys_content_t:s0      html
```

Checking and Changing SELinux Mode at Runtime

Use `getenforce` to see the current mode:

```
getenforce
```

Example:

```
[root@host ~]# getenforce
Enforcing
```

Use `setenforce` to switch between **Enforcing** and **Permissive**:

```
setenforce 0          # Permissive
setenforce 1          # Enforcing
```

or

```
setenforce Permissive
setenforce Enforcing
```

Example:

```
[root@host ~]# setenforce 0
[root@host ~]# getenforce
Permissive
[root@host ~]# setenforce Enforcing
[root@host ~]# getenforce
Enforcing
```

Boot-time Control with Kernel Parameters

You can influence SELinux at boot with kernel arguments:

- `enforcing=1` → start in enforcing mode
- `enforcing=0` → start in permissive mode
- `selinux=1` → enable SELinux
- `selinux=0` → completely disable SELinux (RHEL 9 requires this to truly turn SELinux off)

Recommendation:
 If you switch from **Permissive** to **Enforcing**, reboot the system. This ensures all services start under confined conditions from the beginning of the boot process.

Configuring Default SELinux Mode

Persistent SELinux configuration is stored in:
`/etc/selinux/config`

Typical example:

```
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#   enforcing - SELinux security policy is enforced.
#   permissive - SELinux prints warnings instead of enforcing.
#   disabled   - No SELinux policy is loaded.

SELINUX=enforcing

# SELINUXTYPE= can take one of these three values:
#   targeted - Targeted processes are protected.
#   minimum  - A modification of targeted, protecting only selected processes.
#   mls      - Multi Level Security protection.
SELINUXTYPE=targeted
```

The system reads this file at boot and starts SELinux in the selected mode and policy type.

Note on Older Behavior

In previous Fedora/RHEL builds:

- `SELINUX=disabled` in `/etc/selinux/config` would fully disable SELinux.
- On newer RHEL 9 behavior, to **truly** disable SELinux, you must use `selinux=0` on the kernel command line.

To permanently add `selinux=0` to all kernels:

```
grubby --update-kernel ALL --args selinux=0
```

To revert and re-enable SELinux:

```
grubby --update-kernel ALL --remove-args selinux
```

Kernel boot arguments like `selinux=` and `enforcing=` **override** the settings in `/etc/selinux/config`.

References

- `getenforce(8)` - display current SELinux mode
- `setenforce(8)` - switch between enforcing and permissive
- `selinux_config(5)` - details about `/etc/selinux/config`

Control SELinux File Contexts

Managing Default Labels with `semanage fcontext` and Applying Them Using `restorecon`

Objectives

- Understand how SELinux assigns default contexts to files and directories
- Define custom file context rules using `semanage fcontext`
- Apply correct SELinux labels to files using `restorecon`
- Identify when manual labeling with `chcon` is appropriate

How SELinux Assigns Initial Contexts

Every file, directory, process, and port on the system has an SELinux **context**, which determines how the SELinux policy interacts with that resource.
SELinux keeps its file labeling rules in:
`/etc/selinux/targeted/contexts/files/`

How new files receive labels

1. **If a path matches an existing policy rule:**
   → The new file receives that rule's context automatically.
2. **If no specific policy matches the path:**
   → The file inherits the context of the **parent directory**.

This ensures that *all* files are labeled, even in custom paths.

How copying vs. moving affects context

- **Copy (`cp`)**
  - A new inode is created.
  - The context is determined by the **destination's labeling policy**.
  - You may use:
    - `cp -p` → preserve all metadata
    - `cp --preserve=context` → preserve only SELinux context

- **Move (`mv`)**
  - If the move occurs within the same filesystem, **no new inode** is created.
  - The context **stays the same**, unless you explicitly override it with:
    - `mv -Z`

After moving or copying files, always verify the context and correct it if needed.

Example: How Context Changes When Moving vs. Copying

Create two files in `/tmp`. They inherit the `user_tmp_t` context:

```
touch /tmp/file1 /tmp/file2
ls -Z /tmp/file*
```

Output:

```
unconfined_u:object_r:user_tmp_t:s0 /tmp/file1
unconfined_u:object_r:user_tmp_t:s0 /tmp/file2
```

Check the context of `/var/www/html`:

```
ls -Zd /var/www/html/
```

Output:

```
system_u:object_r:httpd_sys_content_t:s0 /var/www/html/
```

Move and copy the test files:

```
mv /tmp/file1 /var/www/html/
cp /tmp/file2 /var/www/html/
ls -Z /var/www/html/file*
```

Result:

```
/var/www/html/file1 → keeps original user_tmp_t
/var/www/html/file2 → inherits httpd_sys_content_t
```

Changing SELinux Contexts

SELinux contexts can be modified using three major tools:

1. **`semanage fcontext`**
   - Defines or edits policy rules for default contexts
1. **`restorecon`**
   - Applies the policy-defined label to the filesystem
1. **`chcon`**
   - Temporarily changes a file's context (not persistent across relabel operations)

Best practice

Use:

```
semanage fcontext  → define rule
restorecon         → apply rule
```

Avoid relying on `chcon` for long-term labeling.

Why?

If the system is relabeled (for example with `restorecon -R /` or during boot),
any manual context set by `chcon` will be overwritten unless it matches policy.

Example: How `restorecon` Overrides `chcon`

Create a directory:

```
mkdir /virtual
ls -Zd /virtual
```

Initial label:

```
default_t
```

Manually change its type:

```
chcon -t httpd_sys_content_t /virtual
ls -Zd /virtual
```

Now run:

```
restorecon -v /virtual
```

SELinux resets it back:

```
Relabeled /virtual from ...:httpd_sys_content_t:s0 to ...:default_t:s0
```

Defining SELinux Default Context Rules

The **`semanage fcontext`** command manages the context rules that assign default labels.
List all SELinux file context policies:

```
semanage fcontext -l
```

Rules use extended regular expressions.
The most common pattern is:

```
/dir(/.*)?
```

This matches:

- the directory itself
- all files/subdirectories beneath it
- and any nested content

Example rule:

```
/var/www/cgi-bin(/.*)?    all files    system_u:object_r:ht-
tpd_sys_script_exec_t:s0
```

This means:
All content in `/var/www/cgi-bin/` and its descendants is labeled `ht-tpd_sys_script_exec_t`.

Useful `semanage fcontext` Options

Option	Description
`-a`	Add a new context rule
`-d`	Delete a context rule
`-l`	List existing rules

Full Workflow Example: Fixing Contexts in `/var/www`

Check current contexts:

```
ls -Z /var/www/html/file*
```

Example:

```
file1 → user_tmp_t
file2 → httpd_sys_content_t
```

Check policy definition:

```
semanage fcontext -l
```

Look for:

```
/var/www(/.*)?   system_u:object_r:httpd_sys_content_t:s0
```

Apply default labeling recursively:

```
restorecon -Rv /var/www/
```

Result:

```
file1 is relabeled to httpd_sys_content_t
file2 remains correct
```

Example: Adding a New Custom Context Rule

Create a directory with default context:

```
mkdir /virtual
touch /virtual/index.html
ls -Zd /virtual
ls -Z /virtual
```

Add a policy rule:

```
semanage fcontext -a -t httpd_sys_content_t '/virtual(/.*)?'
```

Apply the new rule:

```
restorecon -RFvv /virtual
```

Now:
/virtual and index.html are labeled httpd_sys_content_t

Check custom rules only:
```
semanage fcontext -l -C
```

Output shows your added entry:
```
/virtual(/.*)?  all files  system_u:object_r:httpd_sys_content_t:s0
```

References

- chcon(1)
- restorecon(8)
- semanage(8)
- semanage-fcontext(8)

Adjust SELinux Policy with Booleans

Fine-tuning SELinux Behavior for Applications

Objectives

- Enable or disable optional SELinux policy behavior using `setsebool`
- Make Boolean changes persistent with `setsebool -P`
- Inspect Boolean values and descriptions using `semanage boolean -l`
- Use SELinux man pages (ending in `_selinux`) to understand service-specific options

What Are SELinux Booleans?

Every application with a targeted SELinux policy may include **optional behaviors**. These optional behaviors allow administrators to customize what the service can or cannot do without rewriting policy rules.
SELinux **Booleans** are switches that turn these optional behaviors **on or off**.

- They provide a safe, supported way to adjust SELinux behavior.
- Booleans are defined by application developers as part of the service's targeted policy.
- Service-specific SELinux options are documented in `<service>_selinux` man pages.
  - Example:
    - `man httpd` (service documentation)

- `man httpd_selinux` (SELinux rules, contexts, and Booleans for Apache)

The package containing these policy documents is:
`selinux-policy-doc`

Listing and Managing SELinux Booleans

View all Booleans and their current states:

```
getsebool -a
```

Example output:
```
abrt_anon_write --> off
abrt_handle_event --> off
abrt_upload_watch_anon_write --> on
...
```

Temporarily enable or disable a Boolean:

```
setsebool <boolean> on|off
```

Make a Boolean persistent across reboots:

```
setsebool -P <boolean> on|off
```

Only root (or an SELinux-equivalent privileged role) may modify Boolean values.

Example Boolean: httpd_enable_homedirs

Apache (httpd) has a Boolean named **`httpd_enable_homedirs`.**
By default, Apache is **not allowed** to access user home directories.
Home directories are normally used locally or served through network file protocols like NFS—not through HTTPS.
Check its status:
```
getsebool httpd_enable_homedirs
```

Output:
```
httpd_enable_homedirs --> off
```

If enabled, Apache may serve content from directories labeled with the `user_home_dir_t` type, allowing users to access their home directory files via a web browser.

Understanding Boolean Persistence

A Boolean has:

- A **current value** (applied immediately)
- A **default value** (persistent, loaded at boot)

View persistent and current values together:

```
semanage boolean -l
```

Example (filtered for the httpd Boolean):

```
semanage boolean -l | grep httpd_enable_homedirs
httpd_enable_homedirs   (off , off)   Allow httpd to enable homedirs
```

Change the current value only (not persistent):

```
setsebool httpd_enable_homedirs on
```

Check again:

```
semanage boolean -l | grep httpd_enable_homedirs
```

Output:

```
httpd_enable_homedirs   (on , off)   Allow httpd to enable homedirs
```

- **First value** = current
- **Second value** = default from policy

You can also confirm the running value:

```
getsebool httpd_enable_homedirs
```

Listing Only Modified Booleans

To list **only** Booleans whose runtime values differ from their default boot values:

```
semanage boolean -l -C
```

Example output:

```
SELinux boolean
 State        Default      Description
httpd_enable_homedirs   (on , off)   Allow httpd to enable homedirs
```

This is very useful for change tracking.

Making Boolean Changes Persistent

To permanently modify a Boolean:

```
setsebool -P httpd_enable_homedirs on
```

Now view it again:

```
semanage boolean -l | grep httpd_enable_homedirs
```

Output:

```
httpd_enable_homedirs   (on , on)   Allow httpd to enable homedirs
```

The `-C` option still lists it because it compares against the system's **initial boot default**, not the updated persistent setting.

References

- booleans(8)
- getsebool(8)
- setsebool(8)
- semanage(8)
- semanage-boolean(8)

Investigate and Resolve SELinux Issues

Using SELinux Logs and `sealert` to Diagnose Access Denials

Objectives

- Use SELinux logging and analysis tools to diagnose access problems
- Interpret SELinux AVC (Access Vector Cache) denials
- Use the `sealert` tool for readable explanations and recommended solutions

Understanding SELinux Troubleshooting Basics

When SELinux blocks an action, it logs detailed information that can help identify the issue.
Before making policy changes, it's important to understand how SELinux behaves:

Key concepts

- **SELinux uses targeted policies** that define exactly which interactions are allowed.
- Each rule links:
  - a *process label* (domain)
  - a *resource label* (file/port type)
  - an *allowed action* (for example: read, write, getattr, name_bind)
- If an action is **not explicitly allowed**, SELinux **denies** it.
- Every denial is logged with enough context to diagnose the problem.

When SELinux issues usually appear

Most denials happen when:

- Files have **incorrect contexts** (common after copying/moving files)
- Administrators place files in locations not expected by policy
- A service requires an optional SELinux Boolean that is disabled
- A custom or newly installed application lacks a complete SELinux policy

Important:
For common Red Hat services with existing policies, SELinux rarely causes problems unless something is misconfigured.
Before adjusting SELinux policy, always verify:

1. The service has the **correct file labels**
2. Its `_selinux` manual page does not require enabling a Boolean
3. The process is running in the correct SELinux domain

Monitoring SELinux Violations

SELinux logs access denials as **AVC messages**.
 These logs appear in:
`/var/log/audit/audit.log`

The **setroubleshoot-server** package provides the `setroubleshoot` service, which:

- Watches for AVC denials
- Logs summarized warnings to `/var/log/messages`
- Provides a UUID for each denial event
- Suggests running `sealert` for full details

View the full report for a specific event:

```
sealert -l <UUID>
```

View all logged events:

```
sealert -a /var/log/audit/audit.log
```

Example: SELinux Blocking Apache Content Access

Create a file in `/root`, move it into Apache's content directory, and attempt to access it:

```
touch /root/mypage
mv /root/mypage /var/www/html
systemctl start httpd
curl http://localhost/mypage
```

Result:

```
403 Forbidden
You don't have permission to access this resource.
```

This happens because the file has an **incorrect SELinux context**.

Inspect the AVC entry:

```
tail /var/log/audit/audit.log
tail /var/log/messages
```

Example output snippet:

```
avc: denied { getattr } for pid=2332 comm="httpd" path="/var/www/html/mypage"
...
SELinux is preventing httpd from getattr access. Run: sealert -l <UUID>
```

The `sealert` command provides a human-readable explanation.

Using `sealert` to Diagnose the Issue

Run the suggested command:
```
sealert -l 95f41f98-6b56-45bc-95da-ce67ec9a9ab7
```

Output includes:
- The process label (example: `httpd_t`)
- The file label (example: `admin_home_t`)
- The denied permission
- Suggested solutions

Example suggestions from sealert

1. Correct file label (high confidence)
```
restorecon -v /var/www/html/mypage
```

2. Generate a custom policy (low confidence)
(sealert offers this only when needed)
```
ausearch -c 'httpd' --raw | audit2allow -M my-httpd
semodule -X 300 -i my-httpd.pp
```

Important

Confidence ratings indicate how likely a suggestion is correct—but they are not guarantees.
If the underlying issue is a **wrong file location**, simply applying a new label is not the right solution.
 Correct fix: move file to the appropriate directory and apply the default context with:
```
restorecon -R <directory>
```

Searching for AVC Events Manually

Use `ausearch` to filter AVC denials:
```
ausearch -m AVC -ts recent
```

Example output includes:
- SELinux domain
- Target context
- Denied action
- System call information

This is useful for reviewing recent or historical events.

Troubleshooting SELinux with the RHEL Web Console

The RHEL Web Console includes an SELinux module:

1. Click **SELinux** in the left panel
2. Review current **enforcement mode**
3. Look under **SELinux access control errors** for a list of issues
4. Click the “>`” icon to see full details
5. Select **solution details** for step-by-step remediation
6. Apply the suggested fix directly from the interface

When all issues are resolved, the interface displays:

```
No SELinux alerts
```

References

- `sealert(8)` man page